

# Programmation Impérative 4

Toutes les fonctions avec implémentations · L2 N4 info-DL · Institut Galilée · Paris 13

## 1. Structures de données

### struct binary\_tree (BT.h)

```
typedef struct binary_tree {
    item label; /* étiquette du nœud */
    struct binary_tree *left; /* fils gauche */
    struct binary_tree *right; /* fils droit */
} *link;
```

### struct node\_Q + struct queue (queue\_link.h)

```
struct node_Q { link value; struct node_Q *next; };
struct queue { struct node_Q *first; /* sortie */
              struct node_Q *last; /* entrée */ };
```

### struct stack (stack\_link.h)

```
struct stack { int top; link *contents; };
```

### struct llrbt\_node (llrbt.h — TP5)

```
typedef struct llrbt_node {
    item label; int color; /* RED=1, BLACK=0 */
    struct llrbt_node *left, *right;
} *llrbt_link;
#define RED 1
#define BLACK 0
```

### Macros item.h

```
typedef char item; /* TP1-4: char | TP5: int */
#define less(A,B) (key(A) < key(B))
#define eq(A,B) (!less(A,B) && !less(B,A))
#define key(A) (A)
#define display(A) { printf("%c ", key(A)); }
```

## 2. binary\_tree.c

### empty\_tree()

Retourne NULL (arbre vide).  $O(1)$

```
link empty_tree() {  
    return NULL;  
}
```

### cons\_binary\_tree(item h, const link t1, const link t2)

Alloue un nœud label=h, left=t1, right=t2.  $O(1)$

```
link cons_binary_tree(item h, const link t1, const link t2) {  
    link tree = malloc(sizeof(struct binary_tree));  
    if (!tree) return NULL;  
    tree->label = h; tree->left = t1; tree->right = t2;  
    return tree;  
}
```

### delete\_binary\_tree(link \*bt)

Libère récursivement (postordre : gauche, droite, racine). Met \*bt = NULL.  $O(n)$

```
void delete_binary_tree(link *bt) {  
    if (!bt || *bt == NULL) return;  
    delete_binary_tree(&(*bt)->left);  
    delete_binary_tree(&(*bt)->right);  
    free(*bt);  
    *bt = NULL;  
}
```

### left(bt)

Retourne bt->left. Requiert bt non vide.  $O(1)$

```
link left(const link bt) { return bt->left; }
```

### right(bt)

Retourne bt->right. Requiert bt non vide.  $O(1)$

```
link right(const link bt) { return bt->right; }
```

### get\_binary\_tree\_root(bt)

Retourne bt->label. Requiert bt non vide.  $O(1)$

```
item get_binary_tree_root(const link bt) { return bt->label; }
```

### is\_empty\_binary\_tree(bt)

Retourne 1 si bt == NULL, 0 sinon.  $O(1)$

```
int is_empty_binary_tree(const link bt) { return !bt; }
```

### size\_binary\_tree(bt)

Nombre de nœuds. Récursif : 0 si vide, sinon 1 + size(g) + size(d).  $O(n)$

```
int size_binary_tree(const link bt) {  
    if (is_empty_binary_tree(bt)) return 0;  
    return 1 + size_binary_tree(bt->left)  
    + size_binary_tree(bt->right);  
}
```

### height\_binary\_tree(bt)

Hauteur : -1 si vide, 0 si feuille, sinon  $1 + \max(hg, hd)$ .  $O(n)$

```
int height_binary_tree(const link bt) {
    if (is_empty_binary_tree(bt)) return -1;
    int hl = height_binary_tree(bt->left);
    int hr = height_binary_tree(bt->right);
    return 1 + (hl > hr ? hl : hr);
}
```

### print\_label(bt)

Affiche  $bt \rightarrow label$  via  $display()$ . Ne fait rien si vide.  $O(1)$

```
void print_label(const link bt) {
    if (!is_empty_binary_tree(bt)) display(bt->label);
}
```

### show\_binary\_tree(bt, lvl)

Arbre tourné  $-90^\circ$  (droite en haut). Appel initial :  $lvl=0$ .  $O(n)$

```
void show_binary_tree(const link bt, int lvl) {
    if (is_empty_binary_tree(bt)) return;
    show_binary_tree(bt->right, lvl + 1);
    for (int i = 0; i < lvl; i++) printf(" ");
    printf("%c\n", bt->label);
    show_binary_tree(bt->left, lvl + 1);
}
```

### traverse\_preorder\_binary\_tree(bt, visit)

Parcours récursif PRÉORDRE : Racine  $\rightarrow$  Gauche  $\rightarrow$  Droite.  $O(n)$

```
void traverse_preorder_binary_tree(link bt, void (*visit)(link)) {
    if (is_empty_binary_tree(bt)) return;
    (*visit)(bt);
    traverse_preorder_binary_tree(bt->left, visit);
    traverse_preorder_binary_tree(bt->right, visit);
}
```

### traverse\_inorder\_binary\_tree(bt, visit)

Parcours récursif INFIXE : Gauche  $\rightarrow$  Racine  $\rightarrow$  Droite.  $O(n)$

```
void traverse_inorder_binary_tree(link bt, void (*visit)(link)) {
    if (is_empty_binary_tree(bt)) return;
    traverse_inorder_binary_tree(bt->left, visit);
    (*visit)(bt);
    traverse_inorder_binary_tree(bt->right, visit);
}
```

### traverse\_postorder\_binary\_tree(bt, visit)

Parcours récursif POSTORDRE : Gauche  $\rightarrow$  Droite  $\rightarrow$  Racine.  $O(n)$

```
void traverse_postorder_binary_tree(link bt, void (*visit)(link)) {
    if (is_empty_binary_tree(bt)) return;
    traverse_postorder_binary_tree(bt->left, visit);
    traverse_postorder_binary_tree(bt->right, visit);
    (*visit)(bt);
}
```

### traverse\_preorder\_it\_BT(bt, visit)

Préordre ITÉRATIF avec pile. Empile droite puis gauche (LIFO → gauche en premier).  $O(n)$

```
void traverse_preorder_it_BT(link bt, void (*visit)(link)) {
    if (is_empty_binary_tree(bt)) return;
    struct stack *s = init_stack(size_binary_tree(bt));
    push_stack(s, bt);
    while (!is_empty_stack(s)) {
        link node = pop_stack(s);
        (*visit)(node);
        if (!is_empty_binary_tree(node->right)) push_stack(s, node->right);
        if (!is_empty_binary_tree(node->left)) push_stack(s, node->left);
    }
    delete_stack(&s);
}
```

### traverse\_inorder\_it\_BT(bt, visit)

Infixe ITÉRATIF avec pile + pointeur current. Descend gauche, remonte, part droite.  $O(n)$

```
void traverse_inorder_it_BT(link bt, void (*visit)(link)) {
    struct stack *s = init_stack(size_binary_tree(bt) + 1);
    link current = bt;
    while (!is_empty_binary_tree(current) || !is_empty_stack(s)) {
        while (!is_empty_binary_tree(current)) {
            push_stack(s, current);
            current = current->left;
        }
        current = pop_stack(s);
        (*visit)(current);
        current = current->right;
    }
    delete_stack(&s);
}
```

### traverse\_level\_BT(bt, visit)

Parcours en LARGEUR (BFS) avec file FIFO. Niveau par niveau.  $O(n)$

```
void traverse_level_BT(link bt, void (*visit)(link)) {
    if (is_empty_binary_tree(bt)) return;
    struct queue *q = init_queue();
    enqueue(q, bt);
    while (!is_empty_queue(q)) {
        link node = dequeue(q);
        (*visit)(node);
        if (!is_empty_binary_tree(node->left)) enqueue(q, node->left);
        if (!is_empty_binary_tree(node->right)) enqueue(q, node->right);
    }
    delete_queue(&q);
}
```

## 3. stack\_link.c — Pile

### init\_stack(maxTop)

Alloue pile de capacité maxTop. Initialise top=0 et le tableau contents.  $O(1)$

```
struct stack *init_stack(int maxTop) {
    struct stack *s = malloc(sizeof(struct stack));
    s->top = 0;
    s->contents = malloc(maxTop * sizeof(link));
    return s;
}
```

### delete\_stack(struct stack \*\*s)

Libère contents puis s. Met \*s à NULL.  $O(1)$

```
void delete_stack(struct stack **s) {
    free((*s)->contents);
    free(*s);
    *s = NULL;
}
```

### is\_empty\_stack(s)

Retourne 1 si s->top == 0, 0 sinon.  $O(1)$

```
int is_empty_stack(const struct stack *s) { return (s->top == 0); }
```

### push\_stack(s, val)

Empile val : contents[top++] = val. Ne vérifie pas le débordement.  $O(1)$

```
void push_stack(struct stack *s, link val) { s->contents[s->top++] = val; }
```

### pop\_stack(s)

Dépile : retourne contents[--top]. Requiert pile non vide.  $O(1)$

```
link pop_stack(struct stack *s) { return s->contents[--s->top]; }
```

## 4. queue\_link.c — File

### new\_node\_Q(h)

Alloque un maillon avec *value=h* et *next=NULL*.  $O(1)$

```
struct node_Q *new_node_Q(link h) {
    struct node_Q *nq = malloc(sizeof(struct node_Q));
    nq->value = h; nq->next = NULL;
    return nq;
}
```

### delete\_node\_Q(struct node\_Q \*\*nq)

Libère le maillon \*nq. Met \*nq à NULL.  $O(1)$

```
void delete_node_Q(struct node_Q **nq) {
    free(*nq); *nq = NULL;
}
```

### init\_queue()

Alloque une file vide (*first = last = NULL*).  $O(1)$

```
struct queue *init_queue() {
    struct queue *q = malloc(sizeof(struct queue));
    q->first = NULL; q->last = NULL;
    return q;
}
```

### delete\_queue(struct queue \*\*q)

Libère tous les maillons un à un, puis la file. Met \*q à NULL.  $O(n)$

```
void delete_queue(struct queue **q) {
    while (!is_empty_queue(*q)) {
        struct node_Q *nq = (*q)->first;
        (*q)->first = nq->next;
        delete_node_Q(&nq);
    }
    free(*q); *q = NULL;
}
```

### is\_empty\_queue(q)

Retourne 1 si *q->first == NULL*, 0 sinon.  $O(1)$

```
int is_empty_queue(const struct queue *q) { return (NULL == q->first); }
```

### enqueue(q, h)

Ajoute *h* en fin de file via *q->last*.  $O(1)$

```
void enqueue(struct queue *q, link h) {
    struct node_Q *nq = new_node_Q(h);
    if (is_empty_queue(q)) q->first = nq;
    else q->last->next = nq;
    q->last = nq;
}
```

### dequeue(q)

*Retire et retourne q->first. Met q->last=NULL si la file devient vide. Requiert file non vide. O(1)*

```
link dequeue(struct queue *q) {
    struct node_Q *nq = q->first;
    link res = nq->value;
    q->first = nq->next;
    if (NULL == nq->next) q->last = NULL;
    delete_node_Q(&nq);
    return res;
}
```

## 5. Expressions arithmétiques — TP2/TP3

Notation préfixe : opérateur avant les opérandes. Ex :  $(3*4)+5 \rightarrow + * 3 4 5$

Arbre syntaxique : opérateurs = nœuds internes, chiffres = feuilles.

### is\_operator(char c)

Retourne 1 si c est +, -, \*, /.  $O(1)$

```
int is_operator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/');
}
```

### parse\_expr(const char \*expr, int \*pos)

Construit l'arbre depuis la chaîne préfixe. pos avance globalement à travers tous les appels.  $O(n)$

```
link parse_expr(const char *expr, int *pos) {
    while (expr[*pos] == ' ') (*pos)++;
    char c = expr[*pos]; (*pos)++;
    if (is_operator(c)) {
        link g = parse_expr(expr, pos);
        link d = parse_expr(expr, pos);
        return cons_binary_tree(c, g, d);
    }
    return cons_binary_tree(c, empty_tree(), empty_tree());
}
```

### eval\_tree(const link bt)

Évalue l'arbre (parcours POSTORDRE). Feuille : c - '0'. Nœud : eval(g) OP eval(d).  $O(n)$

```
int eval_tree(const link bt) {
    if (is_empty_binary_tree(bt)) return 0;
    char c = get_binary_tree_root(bt);
    if (is_operator(c)) {
        int vg = eval_tree(bt->left), vd = eval_tree(bt->right);
        switch (c) {
            case '+': return vg+vd; case '-': return vg-vd;
            case '*': return vg*vd; case '/': return vg/vd;
        }
    }
    return c - '0';
}
```

### tree\_to\_expr(const link bt, char \*buf, int \*pos)

Reconstruit la chaîne PRÉFIXE (parcours PRÉORDRE : racine, espace, gauche, espace, droite).  $O(n)$

```
void tree_to_expr(const link bt, char *buf, int *pos) {
    if (is_empty_binary_tree(bt)) return;
    buf[(*pos)++] = get_binary_tree_root(bt);
    if (!is_empty_binary_tree(bt->left)) {
        buf[(*pos)++] = ' '; tree_to_expr(bt->left, buf, pos);
    }
    if (!is_empty_binary_tree(bt->right)) {
        buf[(*pos)++] = ' '; tree_to_expr(bt->right, buf, pos);
    }
}
```



### tree\_to\_infix(const link bt, char \*buf, int \*pos)

Reconstruit la chaîne INFIXE avec parenthèses (parcours INFIXE). Ex :  $((3 * 4) + 5)$ .  $O(n)$

```
void tree_to_infix(const link bt, char *buf, int *pos) {
    if (is_empty_binary_tree(bt)) return;
    char c = get_binary_tree_root(bt);
    if (is_operator(c)) {
        buf[(*pos)++] = '(';
        tree_to_infix(bt->left, buf, pos);
        buf[(*pos)++] = ' '; buf[(*pos)++] = c; buf[(*pos)++] = ' ';
        tree_to_infix(bt->right, buf, pos);
        buf[(*pos)++] = ')';
    } else { buf[(*pos)++] = c; }
}
```

## 6. Tri par tas — TP3

Formules pour  $t[lo..hi]$  :  $\text{fils\_gauche}(i)=2(i-lo)+1+lo$  ·  $\text{fils\_droit}(i)=2(i-lo)+2+lo$  ·  $\text{dernier\_interne}=lo+(hi-lo-1)/2$

### swap(int \*a, int \*b)

Échange \*a et \*b via tmp.  $O(1)$

```
void swap(int *a, int *b) {
    int tmp = *a; *a = *b; *b = tmp;
}
```

### print\_array(const int \*t, int n)

Affiche  $t[0..n-1]$  entre crochets.  $O(n)$

```
void print_array(const int *t, int n) {
    printf("[ ");
    for (int i = 0; i < n; i++) printf("%d ", t[i]);
    printf("]\n");
}
```

### print\_segment(const int \*t, int lo, int hi)

Affiche  $t[lo..hi]$  entre crochets.  $O(hi-lo)$

```
void print_segment(const int *t, int lo, int hi) {
    printf("[ ");
    for (int i = lo; i <= hi; i++) printf("%d ", t[i]);
    printf("]\n");
}
```

### fixHeap(int \*t, int lo, int hi, int i)

Fait descendre  $t[i]$  pour restaurer le tas-max. Suppose les sous-arbres de  $i$  déjà valides.  $O(\log n)$

```
void fixHeap(int *t, int lo, int hi, int i) {
    while (1) {
        int fg = 2*(i-lo)+1+lo; /* fils gauche */
        if (fg > hi) break; /* i est une feuille */
        int j=fg, fd=fg+1;
        if (fd<=hi && t[fd]>t[fg]) j=fd; /* j = plus grand fils */
        if (t[i]>=t[j]) break; /* propriété respectée */
        swap(&t[i], &t[j]); i=j;
    }
}
```

### buildHeap(int \*t, int lo, int hi)

Transforme  $t[lo..hi]$  en tas-max. Appelle fixHeap de droite à gauche sur les nœuds internes.  $O(n)$

```
void buildHeap(int *t, int lo, int hi) {
    int last_parent = lo + (hi-lo-1)/2;
    for (int i=last_parent; i>=lo; i--)
        fixHeap(t, lo, hi, i);
}
```

### heap\_sort(int \*t, int lo, int hi)

Trie  $t[lo..hi]$  croissant. Phase 1 : buildHeap. Phase 2 :  $n-1$  fois échange+fixHeap.  $O(n \log n)$

```
void heap_sort(int *t, int lo, int hi) {
    if (lo >= hi) return;
    buildHeap(t, lo, hi);
    int end = hi;
    while (end > lo) {
        swap(&t[lo], &t[end]);
        end--;
        fixHeap(t, lo, end, lo);
    }
}
```

## 7. ABR — Arbres Binaires de Recherche — TP4

Propriété : gauche < racine <= droite. Infixe = ordre croissant.

### search\_BST(link h, item v)

Recherche récursive de v. Retourne le nœud ou NULL.  $O(n)$  pire cas.

```
link search_BST(link h, item v) {
    if (h==NULL) return NULL;
    item t = get_binary_tree_root(h);
    if (eq(v,t)) return h;
    if (less(v,t)) return search_BST(h->left, v);
    else return search_BST(h->right, v);
}
```

### insert\_BST(link h, item v)

Insère v comme feuille (récursif).  $O(n)$  pire cas.

```
link insert_BST(link h, item v) {
    if (h==NULL) return cons_binary_tree(v, NULL, NULL);
    if (less(v, get_binary_tree_root(h)))
        h->left = insert_BST(h->left, v);
    else
        h->right = insert_BST(h->right, v);
    return h;
}
```

### insert\_BST\_it(link h, item v)

Insère v comme feuille (itératif, sans pile). p=parent, x=courant.  $O(n)$  pire cas.

```
link insert_BST_it(link h, item v) {
    link p, x=h;
    if (h==NULL) return cons_binary_tree(v, NULL, NULL);
    while (x!=NULL) { p=x; x=less(v,get_binary_tree_root(x))?x->left:x->right; }
    x = cons_binary_tree(v, NULL, NULL);
    if (less(v,p->label)) p->left=x; else p->right=x;
    return h;
}
```

### select\_BST(link h, int k)

$t = \text{size}(\text{gauche})$ .  $t > k \rightarrow \text{gauche}$  ;  $t < k \rightarrow \text{droite}$   $k-t-1$  ;  $t==k \rightarrow \text{retourne } h$ .  $O(n^2)$  pire cas.

```
link select_BST(link h, int k) {
    if (h==NULL) return NULL;
    int t = size_binary_tree(h->left);
    if (t>k) return select_BST(h->left, k);
    else if (t<k) return select_BST(h->right, k-t-1);
    return h;
}
```

### rotate\_right(link h)

Fait monter le fils gauche.  $O(1)$ .

```
link rotate_right(link h) {
    link x=h->left; h->left=x->right; x->right=h; return x;
}
```

### rotate\_left(link h)

Fait monter le fils droit. Symétrique de rotate\_right.  $O(1)$ .

```
link rotate_left(link h) {
    link x=h->right; h->right=x->left; x->left=h; return x;
}
```

### insert\_BST\_root(link h, item v)

Insère v à la racine (récursif + rotation).  $O(n)$  pire cas.

```
link insert_BST_root(link h, item v) {
    if (h==NULL) return cons_binary_tree(v, NULL, NULL);
    if (less(v, get_binary_tree_root(h))) {
        h->left = insert_BST_root(h->left, v); h=rotate_right(h);
    } else {
        h->right = insert_BST_root(h->right, v); h=rotate_left(h);
    }
    return h;
}
```

### insert\_BST\_root\_it(link h, item v)

Insère v à la racine (itératif). Split via left\_ptr/right\_ptr.  $O(n)$  pire cas.

```
link insert_BST_root_it(link h, item v) {
    link new_node = cons_binary_tree(v, NULL, NULL);
    if (h==NULL) return new_node;
    link *left_ptr=&(new_node->left), *right_ptr=&(new_node->right);
    link cur=h;
    while (cur!=NULL) {
        if (less(v, get_binary_tree_root(cur))) {
            *right_ptr=cur; right_ptr=&(cur->left); cur=cur->left;
        } else {
            *left_ptr=cur; left_ptr=&(cur->right); cur=cur->right;
        }
    }
    *left_ptr=NULL; *right_ptr=NULL;
    return new_node;
}
```

### partition\_BST(link h, int k)

Amène le rang k à la racine.  $t > k \rightarrow \text{partition gauche} + \text{rotateRight}$ ;  $t < k \rightarrow \text{partition droite} + \text{rotateLeft}$ .  $O(n)$

```
link partition_BST(link h, int k) {
    if (h==NULL) return NULL;
    int t=size_binary_tree(h->left);
    if (t>k) { h->left =partition_BST(h->left, k); h=rotate_right(h); }
    else if (t<k) { h->right=partition_BST(h->right,k-t-1); h=rotate_left(h); }
    return h;
}
```

### join\_BST(link L, link R) [static]

Fusionne L et R (tous élt de L < élt de R). Amène le min de R à la racine via partition(R,0).  $O(n)$

```
static link join_BST(link L, link R) {
    if (L==NULL) return R;
    if (R==NULL) return L;
    R=partition_BST(R, 0); /* min de R à la racine (pas de fils gauche) */
    R->left=L;
    return R;
}
```

### delete\_node\_BST(link h, item v)

Supprime v. Trouve le nœud, fusionne L et R avec join\_BST.  $O(n)$

```
link delete_node_BST(link h, item v) {
    if (h==NULL) return NULL;
    item r=get_binary_tree_root(h);
    if (eq(v,r)) { link L=h->left,R=h->right; free(h); return join_BST(L,R); }
    else if (less(v,r)) h->left = delete_node_BST(h->left, v);
    else h->right = delete_node_BST(h->right, v);
    return h;
}
```

### balance\_BST(link h)

Rééquilibre : amène le médian ( $n/2$ ) à la racine, balance récursivement. Hauteur  $\approx \log_2(n)$ .  $O(n^2)$

```
link balance_BST(link h) {
    int n=size_binary_tree(h);
    if (n<=1) return h;
    h=partition_BST(h, n/2);
    h->left = balance_BST(h->left);
    h->right = balance_BST(h->right);
    return h;
}
```

## 8. LLRBT — Left-Leaning Red-Black Tree — TP5

- Propriété ABR préservée.
- Lien rouge : toujours à GAUCHE (jamais à droite).
- Pas deux liens rouges CONSÉCUTIFS.
- Racine toujours NOIRE.
- Même nombre de liens noirs sur tout chemin racine→feuille.

Hauteur  $\leq 2 \cdot \log_2(n)$ . Tout nouveau nœud est créé rouge.

### new\_llrbt\_node(item v)

Alloque nœud color=RED, left=right=NULL.  $O(1)$

```
llrbt_link new_llrbt_node(item v) {
    llrbt_link h=malloc(sizeof(struct llrbt_node));
    h->label=v; h->color=RED; h->left=NULL; h->right=NULL;
    return h;
}
```

### delete\_llrbt(llrbt\_link \*h)

Libère récursivement (postordre). Met \*h à NULL.  $O(n)$

```
void delete_llrbt(llrbt_link *h) {
    if (!h || *h==NULL) return;
    delete_llrbt(&((*h)->left));
    delete_llrbt(&((*h)->right));
    free(*h); *h=NULL;
}
```

### isRed(llrbt\_link h)

Retourne 1 si h != NULL et h->color == RED. NULL → BLACK.  $O(1)$

```
int isRed(llrbt_link h) {
    if (h==NULL) return 0;
    return h->color==RED;
}
```

### rotateLeft(llrbt\_link h)

Fils DROIT rouge → le faire monter. x hérite couleur de h, h devient RED.  $O(1)$

```
llrbt_link rotateLeft(llrbt_link h) {
    llrbt_link x=h->right;
    h->right=x->left; x->left=h;
    x->color=h->color; h->color=RED;
    return x;
}
```

### rotateRight(llrbt\_link h)

Deux rouges consécutifs à gauche → corriger. Symétrique de rotateLeft.  $O(1)$

```
llrbt_link rotateRight(llrbt_link h) {
    llrbt_link x=h->left;
    h->left=x->right; x->right=h;
    x->color=h->color; h->color=RED;
    return x;
}
```

### flipColors(llrbt\_link h)

Deux fils rouges → inverser les 3 couleurs. Fils deviennent noirs, h devient rouge.  $O(1)$

```
void flipColors(llrbt_link h) {
    h->color=!(h->color);
    h->left->color=!(h->left->color);
    h->right->color=!(h->right->color);
}
```

### insert\_LLGBT(llrbt\_link h, item v)

Descente BST puis 3 corrections en remontant DANS CET ORDRE. Forcer racine à BLACK après appel top-level.  $O(\log n)$

```
llrbt_link insert_LLGBT(llrbt_link h, item v) {
    if (h==NULL) return new_llrbt_node(v);
    if (less(v,h->label)) h->left = insert_LLGBT(h->left, v);
    else h->right = insert_LLGBT(h->right, v);
    /* 3 corrections – toujours dans cet ordre : */
    if ( isRed(h->right) && !isRed(h->left)) h=rotateLeft(h);
    if ( isRed(h->left) && isRed(h->left->left)) h=rotateRight(h);
    if ( isRed(h->left) && isRed(h->right)) flipColors(h);
    return h;
}
```

### inorder\_LLGBT(llrbt\_link h, item \*t, int \*idx)

Parcours infixe → remplit t[] en ordre croissant.  $O(n)$

```
void inorder_LLGBT(llrbt_link h, item *t, int *idx) {
    if (h==NULL) return;
    inorder_LLGBT(h->left, t, idx);
    t[(*idx)++]=h->label;
    inorder_LLGBT(h->right, t, idx);
}
```

### show\_llrbt(llrbt\_link h, int lvl)

Affiche l'arbre tourné -90°. R=rouge, .=noir.  $O(n)$

```
void show_llrbt(llrbt_link h, int lvl) {
    if (h==NULL) return;
    show_llrbt(h->right, lvl+1);
    for (int i=0; i<lvl; i++) printf(" ");
    printf("%c%d\n", h->color==RED ? 'R' : '.', h->label);
    show_llrbt(h->left, lvl+1);
}
```



## 9. Complexités & C essentiel

### Tableau des complexités

| Opération                            | Complexité           |
|--------------------------------------|----------------------|
| Rotation gauche ou droite            | $O(1)$               |
| Push / Pop pile                      | $O(1)$               |
| Enfile / Défile file                 | $O(1)$               |
| isRed, flipColors                    | $O(1)$               |
| Parcours complet (n noeuds)          | $O(n)$               |
| size_binary_tree, height_binary_tree | $O(n)$               |
| buildHeap                            | $O(n)$               |
| fixHeap                              | $O(\log n)$          |
| Insert ABR (moy / pire)              | $O(\log n)$ / $O(n)$ |
| Insert LLRBT (garanti)               | $O(\log n)$          |
| select_BST                           | $O(n^2)$ pire        |
| balance_BST                          | $O(n^2)$             |
| heap_sort                            | $O(n \log n)$        |
| Tri par LLRBT                        | $O(n \log n)$        |

### C essentiel

```
/* Allocation */
link t = malloc(sizeof(struct binary_tree));
if (!t) return NULL; /* toujours vérifier */
free(ptr); ptr = NULL; /* éviter le dangling pointer */

/* Lecture ligne entière */
fgets(buf, sizeof(buf), stdin);
int len = strlen(buf);
if (len > 0 && buf[len-1] == '\n') buf[len-1] = '\0'; /* '\0' pas '\n' ! */

/* Pointeur de fonction */
void traverse_inorder(link bt, void (*visit)(link)); /* déclaration */
traverse_inorder(bt, print_label); /* passage */
(*visit)(node); /* appel */

/* Passage par pointeur */
void f(int *pos) { (*pos)++; } /* modifier la variable de l'appelant */
f(&pos); /* appel */

/* Makefile : -c pour .o, -I pour headers, tabulation pour les recettes */
```